

Basic Data Structures: Dynamic Arrays and Amortized Analysis

Neil Rhodes

Department of Computer Science and Engineering
University of California, San Diego

Data Structures
Data Structures and Algorithms

Outline

- 1 Dynamic Arrays
- 2 Amortized Analysis—Aggregate Method
- 3 Amortized Analysis—Banker's Method
- 4 Amortized Analysis—Physicist's Method

Problem: static arrays are static!

```
int my_array[100];
```

Problem: static arrays are static!

```
int my_array[100];
```

Semi-solution: dynamically-allocated arrays:

```
int *my_array = new int[size];
```

Problem: might not know max size when allocating an array

Problem: might not know max size when allocating an array

*All problems in computer science
can be solved by another level of
indirection.*

Problem: might not know max size when allocating an array

All problems in computer science can be solved by another level of indirection.

Solution: *dynamic arrays* (also known as *resizable arrays*)

Idea: store a pointer to a dynamically allocated array, and replace it with a newly-allocated array as needed.

Definition

Dynamic Array:

Abstract data type with the following operations (at a minimum):

*must be constant time

Definition

Dynamic Array:

Abstract data type with the following operations (at a minimum):

- $\text{Get}(i)$: returns element at location i^*

*must be constant time

Definition

Dynamic Array:

Abstract data type with the following operations (at a minimum):

- $\text{Get}(i)$: returns element at location i^*
- $\text{Set}(i, val)$: Sets element i to val^*

*must be constant time

Definition

Dynamic Array:

Abstract data type with the following operations (at a minimum):

- $\text{Get}(i)$: returns element at location i^*
- $\text{Set}(i, val)$: Sets element i to val^*
- $\text{PushBack}(val)$: Adds val to the end

*must be constant time

Definition

Dynamic Array:

Abstract data type with the following operations (at a minimum):

- $\text{Get}(i)$: returns element at location i^*
- $\text{Set}(i, val)$: Sets element i to val^*
- $\text{PushBack}(val)$: Adds val to the end
- $\text{Remove}(i)$: Removes element at location i

*must be constant time

Definition

Dynamic Array:

Abstract data type with the following operations (at a minimum):

- $\text{Get}(i)$: returns element at location i^*
- $\text{Set}(i, val)$: Sets element i to val^*
- $\text{PushBack}(val)$: Adds val to the end
- $\text{Remove}(i)$: Removes element at location i
- $\text{Size}()$: the number of elements

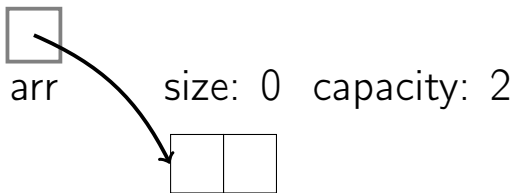
*must be constant time

Implementation

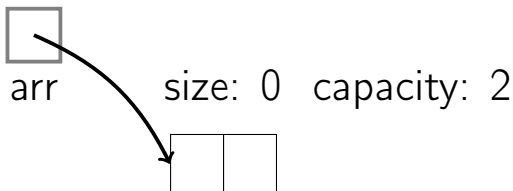
Store:

- `arr`: dynamically-allocated array
- `capacity`: size of the dynamically-allocated array
- `size`: number of elements currently in the array

Dynamic Array Resizing

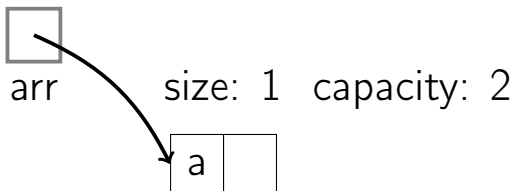


Dynamic Array Resizing



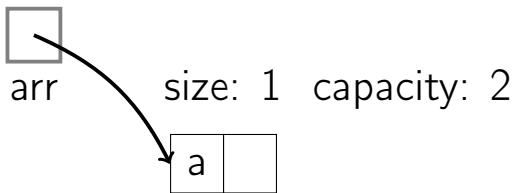
PushBack(a)

Dynamic Array Resizing

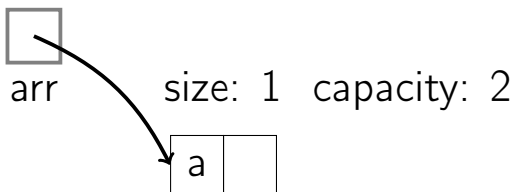


PushBack(a)

Dynamic Array Resizing

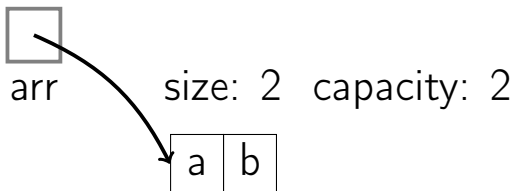


Dynamic Array Resizing



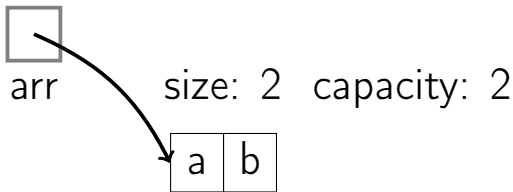
PushBack(b)

Dynamic Array Resizing

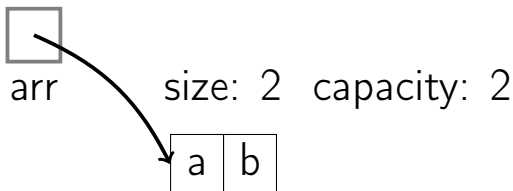


PushBack(b)

Dynamic Array Resizing

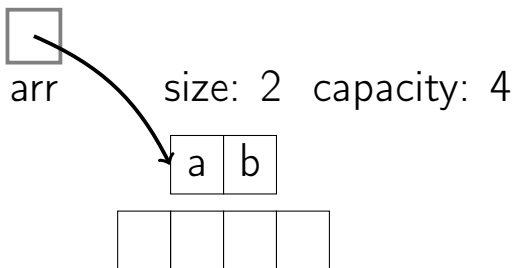


Dynamic Array Resizing



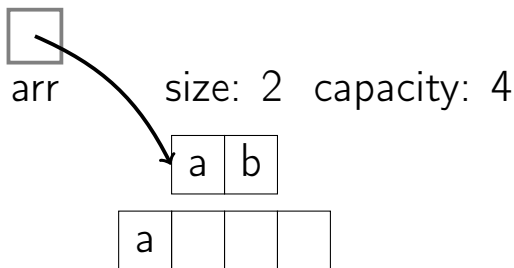
PushBack(c)

Dynamic Array Resizing



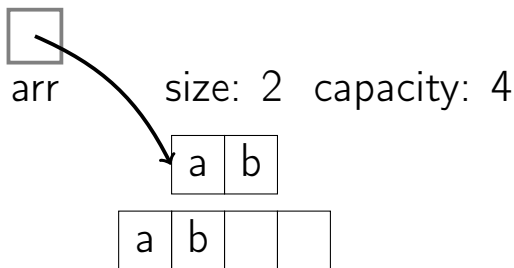
PushBack(c)

Dynamic Array Resizing



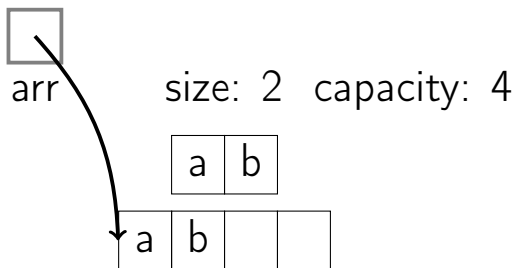
PushBack(c)

Dynamic Array Resizing



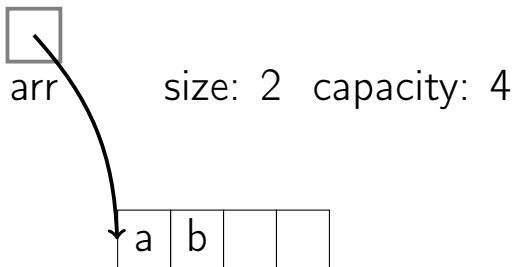
PushBack(c)

Dynamic Array Resizing



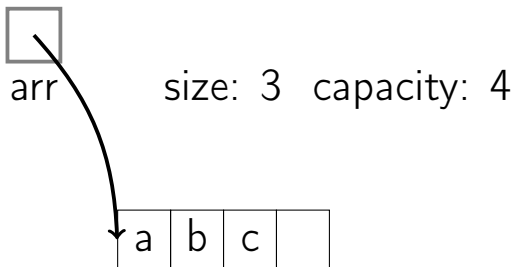
PushBack(c)

Dynamic Array Resizing



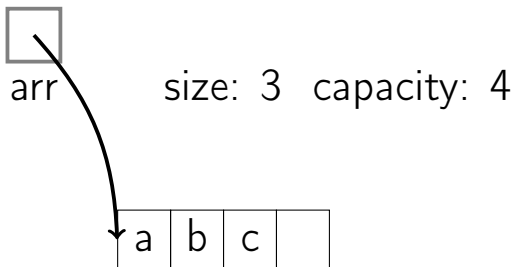
PushBack(c)

Dynamic Array Resizing

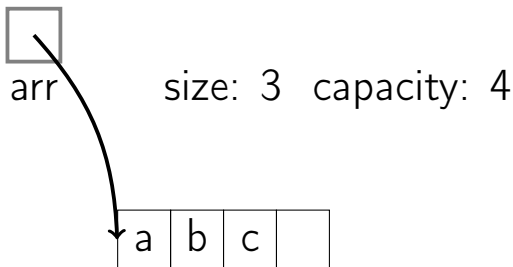


`PushBack(c)`

Dynamic Array Resizing

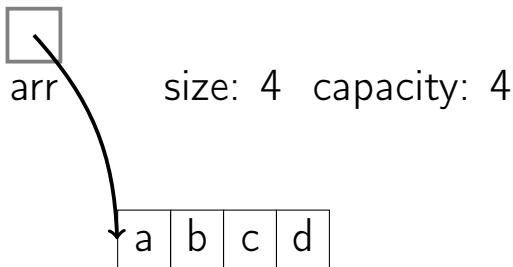


Dynamic Array Resizing



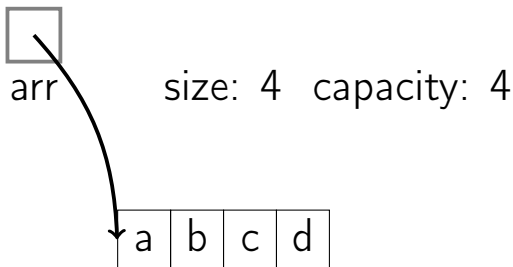
PushBack(d)

Dynamic Array Resizing

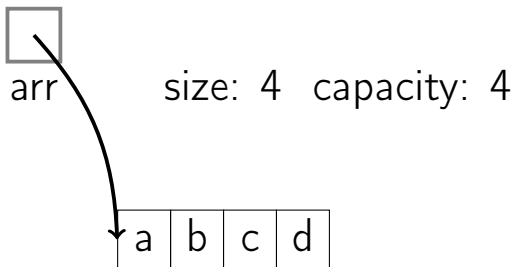


`PushBack(d)`

Dynamic Array Resizing

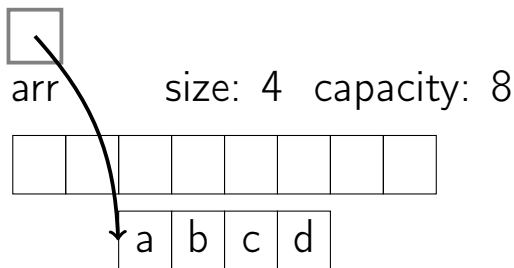


Dynamic Array Resizing



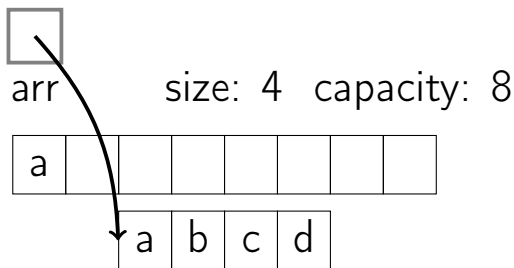
`PushBack(e)`

Dynamic Array Resizing



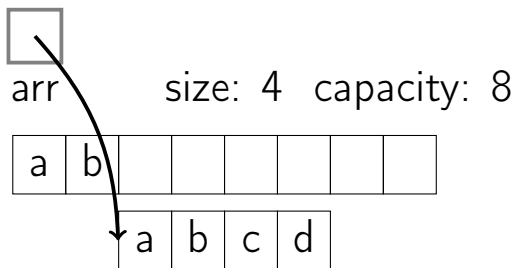
PushBack(e)

Dynamic Array Resizing



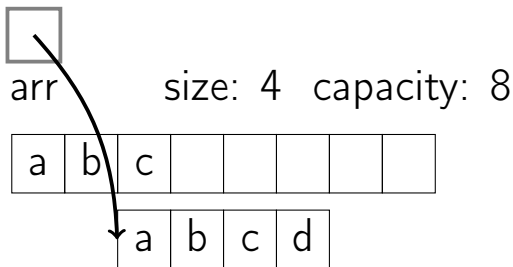
PushBack(e)

Dynamic Array Resizing



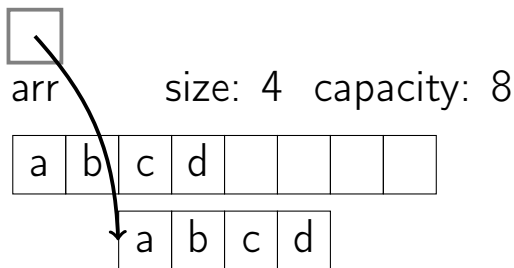
PushBack(e)

Dynamic Array Resizing



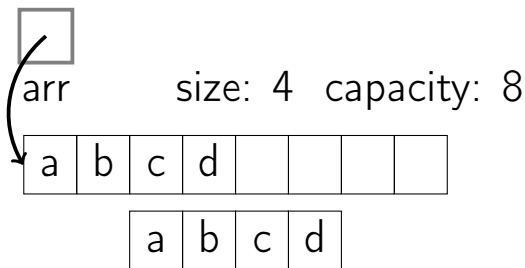
PushBack(e)

Dynamic Array Resizing



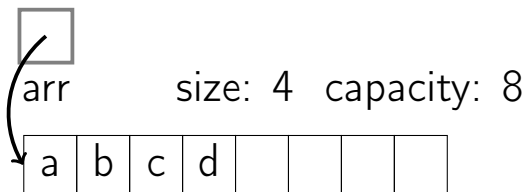
PushBack(e)

Dynamic Array Resizing



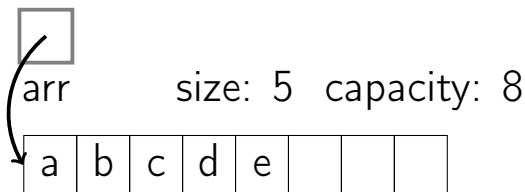
PushBack(e)

Dynamic Array Resizing



PushBack(e)

Dynamic Array Resizing



`PushBack(e)`

Get(*i*)

```
if  $i < 0$  or  $i \geq size$ :
```

```
    ERROR: index out of range
```

```
return arr[i]
```

Set(*i*, *val*)

if $i < 0$ or $i \geq \text{size}$:

 ERROR: index out of range

$\text{arr}[i] = \text{val}$

PushBack(*val*)

```
if size = capacity:  
    allocate new_arr[ $2 \times \textit{capacity}$ ]  
    for i from 0 to size - 1:  
        new_arr[i]  $\leftarrow$  arr[i]  
    free arr  
    arr  $\leftarrow$  new_arr; capacity  $\leftarrow 2 \times \textit{capacity}$   
arr[size]  $\leftarrow$  val  
size  $\leftarrow$  size + 1
```

Remove(*i*)

if $i < 0$ or $i \geq \text{size}$:

 ERROR: index out of range

for j from i to $\text{size} - 2$:

$\text{arr}[j] \leftarrow \text{arr}[j + 1]$

$\text{size} \leftarrow \text{size} - 1$

```
Size()
```

```
return size
```

Common Implementations

- C++: `vector`
- Java: `ArrayList`
- Python: `list` (the only kind of array)

Runtimes

Get(i) | $O(1)$

Runtimes

Get(<i>i</i>)	$O(1)$
Set(<i>i</i> , <i>val</i>)	$O(1)$

Runtimes

Get(i)	$O(1)$
Set(i, val)	$O(1)$
PushBack(val)	$O(n)$

Runtimes

Get(i)	$O(1)$
Set(i, val)	$O(1)$
PushBack(val)	$O(n)$
Remove(i)	$O(n)$

Runtimes

Get(i)	$O(1)$
Set(i , val)	$O(1)$
PushBack(val)	$O(n)$
Remove(i)	$O(n)$
Size()	$O(1)$

Summary

- Unlike static arrays, dynamic arrays can be resized.

Summary

- Unlike static arrays, dynamic arrays can be resized.
- Appending a new element to a dynamic array is often constant time, but can take $O(n)$.

Summary

- Unlike static arrays, dynamic arrays can be resized.
- Appending a new element to a dynamic array is often constant time, but can take $O(n)$.
- Some space is wasted

Summary

- Unlike static arrays, dynamic arrays can be resized.
- Appending a new element to a dynamic array is often constant time, but can take $O(n)$.
- Some space is wasted

Summary

- Unlike static arrays, dynamic arrays can be resized.
- Appending a new element to a dynamic array is often constant time, but can take $O(n)$.
- Some space is wasted—at most half.